Faster Polynomial-Space Quantum Algorithms for Colouring, Domination and Monotone Local Search

Sebastian Senge
Independent Researcher

Abstract

We give quantum algorithms for exponential-time problems on n vertices or elements with bounded error, polynomial space and no quantum random access memory. Our results are these:

- 3-Colouring can be solved in quantum time $O^*(1.3217^{n/2}) \subset O(1.1497^n)$, from the classical algorithm of [28].
- For every $4 \leq k \leq 20$, k-Colouring can be solved in quantum time below the previous best bound in this model to our knowledge, from $O(1.3204^n)$ for $k = 4$ to $O(1.9574^n)$ for $k = 20$.
- Dominating Set can be solved in quantum time $O(1.2235^n)$, Independent Dominating Set in $O(1.1583^n)$, and Edge Dominating Set and Minimum Maximal Matching in $O(1.1501^n)$.
- A quantum form of monotone local search: if the extension problem has a backtracking tree with c^k nodes up to polynomial factors, the problem is solved in quantum time $(2-1/c)^{n/2}$ up to polynomial factors. This gives Feedback Vertex Set in $O(1.3081^n)$, Subset and Group Feedback Vertex Set in $O(1.3229^n)$, Cluster Vertex Deletion in $O(1.2033^n)$, 3-Hitting Set in $O(1.2405^n)$, Proper Interval Vertex Deletion in $O(1.3541^n)$, Multicut on Trees in $O(1.1647^n)$ and Min-Ones d-SAT in $O^*((2-1/d)^{n/2})$. The approximate form gives quantum exponential-time approximations below the quantum exact bounds, such as a 1.1-approximation for Vertex Cover in $O(1.0555^n)$ and for Feedback Vertex Set in $O(1.2161^n)$.
- The exponents for k-colouring with $k \geq 6$ come from an evaluation of the reduction recursion whose values bound the running time of an explicit algorithm, computed in floating point with a margin.

The colouring bounds use the running time of [28] as stated there. To our knowledge, the previous best bounds in this model are $O(1.1528^n)$ for $k = 3$ and $O(1.3231^n)$ to $O(1.9575^n)$ for $4 \leq k \leq 20$ [35], and $O(2^{n/2})$ for Dominating Set [41]. For the other domination problems we know of no stated quantum bound; the walk on the enumeration of maximal independent sets [29] gives $O^*(3^{n/6}) \subset O(1.2010^n)$.

1 Introduction

Deciding whether a graph has a proper colouring with k colours is one of the basic NP-complete problems. For $k = 3$ the fastest classical algorithms are branching algorithms. The algorithm of [5] runs in time $O^*(1.3289^n)$, and [28] improves it to $O^*(1.3217^n)$, with a new choice of the vertices that are coloured by enumeration. For larger k, colouring reduces to 3-colouring through maximal independent sets [26, 8] and through balanced splits of the colour classes.

Quantum search applies to branching algorithms, as noted in [18], and gives the square root of the classical time of [5], so 3-colouring in quantum time $O(1.1528^n)$. The work [35] combines this with the reductions above and gives polynomial-space quantum algorithms without quantum

random access memory for every $k \leq 20$ below $O(2^n)$. Their Theorem 9 takes the exponent of 3-colouring as input. With quantum random access memory, the chromatic number can be computed in time $O(1.7956^n)$ [19], which is outside the model of this paper.

For domination problems, the best polynomial-space classical algorithms are branch and reduce algorithms analysed by measure and conquer [32, 33, 7, 23, 42]. For Dominating Set, a quantum bound of $O(2^{n/2})$ queries was stated in [41]. For many vertex deletion and hitting set problems the best classical bounds come from monotone local search [17], which samples part of a solution at random and completes it with a parameterized algorithm.

Our contribution. We show that the algorithm of [28] is a backtracking tree in the sense of [30]. This gives 3-colouring in quantum time $O^*(1.3217^{n/2})$. We then evaluate the recursion of [35] with this exponent. The evaluation bounds the recursion from above on pieces of a partition, with all size limits rounded up to a grid, and we show that the algorithm can be run with these rounded limits. So the values are upper bounds on the time of an explicit algorithm. Table 1 lists the results. In Section 5 we show that the polynomial-space algorithms of [32, 33, 7] for domination problems are backtracking trees in their decision form, which gives the bounds of Table 2. In Section 6 we give a quantum form of monotone local search, with amplitude amplification over the samples and the walk of [30] on the extension algorithm, and apply it to the problems of Table 3.

k	Previous best bound [35]	This paper	k	Previous best bound [35]	This paper
3	1.1528^n	$\mathbf{1.1497^n}$	12	1.8246^n	$\mathbf{1.8227^n}$
4	1.3231^n	$\mathbf{1.3204^n}$	13	1.8499^n	$\mathbf{1.8487^n}$
5	1.4695^n	$\mathbf{1.4671^n}$	14	1.8580^n	$\mathbf{1.8573^n}$
6	1.5261^n	$\mathbf{1.5238^n}$	15	1.9303^n	$\mathbf{1.9288^n}$
7	1.6511^n	$\mathbf{1.6492^n}$	16	1.9303^n	$\mathbf{1.9288^n}$
8	1.6585^n	$\mathbf{1.6564^n}$	17	1.9303^n	$\mathbf{1.9288^n}$
9	1.7460^n	$\mathbf{1.7445^n}$	18	1.9366^n	$\mathbf{1.9350^n}$
10	1.7775^n	$\mathbf{1.7756^n}$	19	1.9575^n	$\mathbf{1.9568^n}$
11	1.7775^n	$\mathbf{1.7756^n}$	20	1.9575^n	$\mathbf{1.9574^n}$

Table 1: Quantum time for k-colouring with bounded error, polynomial space and no quantum random access memory. All values are rounded up. The previous bounds are the best known to us.

Scope. All rows of Table 1 use the running time of [28] as stated there. That paper is an arXiv paper and, to our knowledge, unrefereed, and its bound comes from a linear program over the structure of its forests, which we do not reproduce. If the algorithm of [5] is used in its place, every row returns to the values of [35]. For the rows $k = 3, 4, 5$, the arithmetic of the closed forms is checked in Lean. The rows $k = 6$ to 20 come from a floating point computation with a margin far above its rounding error, not from interval arithmetic. Numbers of theorems, lemmas and appendices of [35] refer to its arXiv version. Code and Lean files are at `https://github.com/ssenge/LAIF`, in the folders `quantum-graph-colouring` and `quantum-dominating-set`.

2 Preliminaries

For $\delta \in [0, 1]$ let $h(\delta) = -\delta \log \delta - (1 - \delta) \log(1 - \delta)$, with logarithms to base 2. O^* hides factors polynomial in n.

Backtracking trees. A backtracking tree is given by a root instance and a procedure that, in time $\mathrm{poly}(n)$, decides for an instance whether it is a leaf, and if so whether it is marked, or else lists its children. The depth is at most $\mathrm{poly}(n)$. Given a number T computable in time $\mathrm{poly}(n)$, the quantum walk of [30] decides whether a tree with at most T nodes has a marked leaf, with bounded error, in time $O(\sqrt{T}\,\mathrm{poly}(n))$ and polynomial space. Its circuit depends only on n and T, so it can be used inside quantum search.

Maximal independent sets. By [8], an n-vertex graph has at most $I(n,t) = O^*(2^{E(t/n)n})$ maximal independent sets of size t, for a concave piecewise linear function E with $E(1/s) = (\log s)/s$, and they can be listed by a branching algorithm whose number of leaves is bounded by $I(n,t)$ in the form required by Lemma 4 of [35].

3 3-Colouring

Theorem 3.1. *3-Colouring has a quantum algorithm with bounded error, polynomial space and no quantum random access memory, running in time* $O^*(1.3217^{n/2}) \subset O(1.1497^n)$.

Proof. The algorithm of [28] has three kinds of steps.

(i) Reductions and branching rules on the graph, such as the rules on vertices of degree three. Each replaces the instance by polynomially many smaller instances, and the answer is yes if and only if one of them is a yes-instance.

(ii) The choice of a set S of vertices, from its forests, in polynomial time, followed by the colourings of S that the analysis of [28] counts. Each such colouring is one instance of the next step, and the answer is the OR over them. We colour the vertices of S one at a time, in the order of [28], in which each vertex of a tree of the forest comes after its parent. The children of a node are the colours of the next vertex that differ from the colours of its already coloured neighbours, so a vertex after its parent has at most two children, and where [28] colours two vertices together, the node lists the pairs of colours in the same way. A node stores the set S, the order, and the partial colouring, all fixed when S is chosen, so it does not recompute the forests on a partly coloured graph. The leaves of this part are the colourings counted in [28].

(iii) For each colouring of S, a $(3,2)$-constraint satisfaction instance on the remaining vertices, solved by the branching algorithm of [5], whose steps are again of the form (i).

Every node computes its children in polynomial time and stores a description of polynomial size, and the depth is polynomial. So the algorithm is a backtracking tree, with a marked leaf if and only if the graph is 3-colourable. Its number of nodes is at most its running time, so by [28] there is a fixed polynomial p with at most $T_n = p(n)\,1.3217^n$ nodes on every graph with n vertices. The walk of [30] with the bound T_n decides 3-colourability in time $O^*(1.3217^{n/2})$, the same for every input of size n, and $1.3217 \le 1.1497^2$. This is an analogue of Lemma 6 of [35] for the algorithm of [5], with the walk in place of search over leaf indices, which needs no local bound on the number of leaves.

A colouring follows by self-reduction. Add a triangle on three new vertices c_1, c_2, c_3. For each vertex v in turn, merge v with one of c_1, c_2, c_3, and keep the first merge that leaves a 3-colourable graph. The graph has at most $n+3$ vertices throughout, so this costs a factor $O(n)$ and a constant factor in the time. □

4 k-Colouring

4.1 The recursion

Let $\textsc{Col}(G, k, \mu)$ be a procedure that answers yes if the vertex set of G, of size n, splits into k independent sets of size at most μn each, answers no if G is not k-colourable, and may answer either otherwise. Then $\textsc{Col}(G, k, 1)$ decides k-colourability. Calling $\textsc{Col}$ with a larger μ only weakens the first condition, so it is always correct to replace μ by a larger value.

Theorem 9 of [35] gives such a procedure for $k \geq 4$ from two reductions, each with quantum search over its choices.

- **Reduction 1.** Search over the maximal independent sets I of size $t \geq n/k$, and call $\textsc{Col}(G - I, k-1, 1)$. In a colouring with k classes the largest class can be extended to a maximal independent set, which has size at least n/k.
- **Reduction 2.** Fix $2 \leq k' \leq k/2$. Search over the sets S of size t with $k'n/k \leq t \leq \min(n, \mu k' n)$, and answer yes if $\textsc{Col}(G[S], k', \mu n/t)$ and $\textsc{Col}(G - S, k - k', \lfloor t/k' \rfloor/(n-t))$ both answer yes. For $t = n$ the graph $G - S$ is empty and the second call is omitted. If S is the union of the k' largest classes of a colouring with classes of size at most μn, then S has this size, its classes have size at most μn, and the remaining $k - k'$ classes are at most the average t/k' of the k' largest ones.

Let $d_3 = f_3$ be the exponent of 3-colouring, let $d_1 = d_2 = 0$, and for $k \geq 4$ and $\mu \in [1/k, 1]$ let

$$d_k(\mu) = \min\Big(\max_{1 \leq s \leq k} \frac{\log s}{2s} + \Big(1 - \frac{1}{s}\Big) d_{k-1}(1), \tag{1}$$

$$\min_{2 \leq k' \leq k/2} \max_{\delta \in [k'/k,\, \min(1, \mu k')]} \frac{h(\delta)}{2} + \max\Big(\delta\, d_{k'}\big(\min(1, \tfrac{\mu}{\delta})\big), \tag{2}$$

$$(1-\delta)\, d_{k-k'}\big(\min(1, \tfrac{\delta}{k'(1-\delta)})\big)\Big)\Big).$$

Line (1) is the time of Reduction 1: quantum search over $I(n,t)$ sets costs $2^{E(\delta)n/2}$ for $\delta = t/n$, the maximum of $E(\delta)/2 + (1-\delta)d_{k-1}(1)$ over $\delta \in [1/k, 1]$ is attained at some $\delta = 1/s$ by concavity (Corollary 3 of [35]), and $E(1/s) = (\log s)/s$. The term $s = 1$ is 0 and is included only to make the range simple; [35] uses $2 \leq s \leq k$. Line (2) is the time of Reduction 2, with $\binom{n}{t} = O^*(2^{h(\delta)n})$ sets. By Theorem 9 of [35], k-colouring has a quantum algorithm in time $O^*(2^{d_k(1)n})$ with bounded error, polynomial space and no quantum random access memory. The depth of the recursion is at most $k \leq 20$, so the factors from nested search and error reduction stay polynomial.

4.2 Evaluation as an upper bound

The maximum over δ in (2) has no closed form, and $d_{k'}$ and $d_{k-k'}$ are needed at many arguments. We replace both by quantities that the algorithm itself can use.

Fix a grid $\mu_j = j/M$ for $j = 0, \ldots, M$, and let $\lceil \mu \rceil_M$ be the smallest grid point at least μ, for $\mu \leq 1$. For each range $[k'/k, \min(1, \mu_j k')]$, fix a partition into P pieces $[a, b]$. Define $\hat{d}_1 = \hat{d}_2 = 0$, $\hat{d}_3 = f_3$, and for $k \geq 4$ let $\hat{d}_k(\mu_j)$ be the minimum of line (1) with $\hat{d}_{k-1}(1)$ in place of $d_{k-1}(1)$ and of

$$\min_{2 \leq k' \leq k/2} \max_{[a,b]} \frac{\max_{\delta \in [a,b]} h(\delta)}{2} + \max\Big(b\, \hat{d}_{k'}\big(\lceil \min(1, \tfrac{\mu_j}{a}) \rceil_M\big),\ (1-a)\, \hat{d}_{k-k'}\big(\lceil \min(1, \tfrac{b}{k'(1-b)}) \rceil_M\big)\Big), \tag{3}$$

where the argument of $\hat{d}_{k-k'}$ is 1 when $b = 1$.

Proposition 4.1. *For every $k \geq 1$ and grid point $\mu_j \geq 1/k$, there is a quantum procedure* $\textsc{Col}(\cdot, k, \mu_j)$ *with bounded error, polynomial space and no quantum random access memory, in time* $O^*(2^{\hat{d}_k(\mu_j)n})$.

Proof. By induction on k. For $k \leq 2$ colouring takes polynomial time, and for $k = 3$ we use Theorem 3.1 and ignore μ. Let $k \geq 4$. Reduction 1 calls $\textsc{Col}(\cdot, k-1, 1)$, and $1 = \mu_M$ is a grid point, so its time is line (1) with $\hat{d}_{k-1}(1)$.

For Reduction 2 with a fixed k', every size t in the search has $\delta = t/n$ in some piece $[a, b]$. For t in this piece we call

$$\textsc{Col}\big(G[S], k', \lceil \min(1, \mu_j/a) \rceil_M\big) \quad \text{and} \quad \textsc{Col}\big(G - S, k - k', \lceil \min(1, b/(k'(1-b))) \rceil_M\big).$$

These limits are at least the ones of Reduction 2: since $t \geq an$, the classes of $G[S]$ have size at most $\mu_j n \leq (\mu_j/a)\, t$, and since $\delta \leq b$, the remaining classes have size at most $t/k' \leq (b/(k'(1-b)))(n-t)$. A limit of 1 always holds, and it is the one used when $b = 1$, where $G - S$ can still have up to $(1-a)n$ vertices. So the procedure stays correct. By induction the two calls take time $O^*(2^{\hat{d}_{k'}(\cdot)\, t})$ and $O^*(2^{\hat{d}_{k-k'}(\cdot)(n-t)})$ at the stated grid points, and $t \leq bn$ and $n - t \leq (1-a)n$. Quantum search over the $\binom{n}{t} = O^*(2^{h(\delta)n})$ sets S of size t costs a factor $2^{h(\delta)n/2} \leq 2^{n \max_{[a,b]} h/2}$. Summing over the polynomially many t and taking the best k' gives (3). The procedure is run with k and the reduction fixed in advance, and the base case of Theorem 3.1 has a running time that depends only on n, so the circuit is the same for every input of size n. □

A call $\textsc{Col}(G, k, 1)$ decides k-colourability, so $\hat{d}_k(1)$ is an exponent for k-colouring.

4.3 The values

Take $f_3 = \frac{1}{2} \log 1.3217$ from Theorem 3.1.

Rows $k = 3, 4, 5$. For $k \leq 5$ line (1) suffices. Its term for s equals log of $s^{1/(2s)} x^{(s-1)/s}$ with $x = 2^{d_{k-1}(1)}$, and this is at most $\log y$ whenever $s\, x^{2(s-1)} \leq y^{2s}$. With $x = \sqrt{1.3217}$ and $y = 1.32034$, and then with $x = 1.32034$ and $y = 1.4671$, these are polynomial inequalities for $s = 1, \ldots, 5$, all true. So the bases are at most 1.32034 and 1.4671, and $\sqrt{1.3217} \leq 1.1497$.

Rows $k = 6$ to 20. We evaluate (3) with $M = P = 4000$, equal pieces, and add 10^{-9} to each exponent, which is far above the floating point error. Table 1 shows the results, rounded up. The optimal k' at $\mu = 1$ is $\lfloor k/2 \rfloor$ for $k = 6, \ldots, 12$, and $6, 6, 7, 7, 7, 8, 8, 8$ for $k = 13, \ldots, 20$.

As a test of the evaluation, the same computation with the exponent of [5] in place of f_3 gives the values of [35] to within 0.0008 in the base, and in all rows at least these values, as expected from an upper bound.

5 Domination problems

A dominating set of a graph is a set D of vertices such that every vertex is in D or has a neighbour in D. An independent dominating set is a dominating set with no edge inside it, and an edge dominating set is a set of edges such that every edge shares an endpoint with one of them. Minimum Maximal Matching asks for a smallest maximal matching. In each case the decision version asks for a solution of size at most k.

Problem	Previous best bound	This paper
Dominating Set	$2^{n/2}$ quantum [41]; 1.4864^n [23], 1.4969^n [32] classical	$\mathbf{1.2235^n}$
Independent Dominating Set	1.3351^n [6], $2^{0.424n}$ [7] classical; 1.2010^n quantum	$\mathbf{1.1583^n}$
Edge Dominating Set	1.3160^n [42], 1.3226^n [33] classical; 1.2010^n quantum	$\mathbf{1.1501^n}$
Minimum Maximal Matching	1.3160^n classical [42, 33]; 1.2010^n quantum	$\mathbf{1.1501^n}$
Weighted versions of the last two	1.3226^n classical [33]	$\mathbf{1.1501^n}$

Table 2: Domination problems on n vertices. All classical bounds are for polynomial space, and the previous bounds are the best known to us. The quantum bounds are for bounded error, polynomial space and no quantum random access memory, up to polynomial factors. The quantum value 1.2010^n is the walk on the enumeration of maximal independent sets [29], and for edge domination of the minimal vertex covers; to our knowledge it has not been stated. For unweighted graphs, Minimum Maximal Matching and Edge Dominating Set have the same optimum, with a conversion in polynomial time [33].

Theorem 5.1. *With bounded error, polynomial space and no quantum random access memory, Dominating Set has a quantum algorithm in time $O^*(1.4969^{n/2}) \subset O(1.2235^n)$, Independent Dominating Set one in time $O^*(2^{0.212n}) \subset O(1.1583^n)$, and Edge Dominating Set, Minimum Maximal Matching and their weighted versions with integer weights of polynomial bit length have quantum algorithms in time $O^*(1.3226^{n/2}) \subset O(1.1501^n)$.*

Proof. For each problem we show that the decision version of the classical algorithm, with a budget k, is a backtracking tree: each node answers, makes one call with a new budget, or branches into the OR of polynomially many children; it computes them in polynomial time and stores an instance of polynomial size, and the depth is polynomial. The classical analysis bounds the number of leaves of the full search tree by $p(n)\,c^n$ for the classical base c and a fixed polynomial p, and the tree for a budget k is a subtree of it. The depth is at most $2n$ for Dominating Set, since each step removes a set or an element and the rule below removes two sets and two elements and adds one set; at most n for edge domination, since each step lowers the number of undecided vertices; and at most $2n + n^2$ for Independent Dominating Set, counting vertices, free vertices and edges. So the tree has at most $(2n + n^2 + 1)\,p(n)\,c^n$ nodes. The walk of [30] with the bound $T_n = p(n)\,c^n$ then decides the problem in time $O^*(c^{n/2})$, the same for every input of size n. Trying every budget, or a binary search for integer weights, gives the optimum. A solution follows by descending the tree from the root, running the detection on each child and moving to one that contains a marked leaf, and reading the solution off that leaf; with errors amplified to $1/\mathrm{poly}(n)$ per call this costs a polynomial factor.

Dominating Set. Algorithm 4 of [32], in the numbering of its technical report, works on the set cover instance with the closed neighbourhoods as sets over the vertices. If no element is left, it answers yes when $k \geq 0$; if $k \leq 0$ or an element lies in no set, it answers no; if all sets have at most two elements, a maximum matching gives the minimum cover, which it compares with k. Its reduction rules make one call each. An element of frequency one forces its set, with budget $k - 1$. A set contained in another set, and an element whose sets include all sets of another element, are removed, with budget k. The counting rule of [32] takes a set, with budget $k - 1$. The rule for a set $R = \{e_1, e_2\}$ of two elements of frequency two, with second sets R_1 and R_2, replaces R, R_1, R_2

by $(R_1 \cup R_2) \setminus R$ on the elements outside R. Since the subset rules are applied first, $R_1 \neq R_2$ and $R \not\subseteq R_i$. By the argument in [32], the minimum cover then drops by exactly one, so the call has budget $k-1$. Otherwise the algorithm branches on a set S of maximum size, into taking S with budget $k-1$ or discarding S with budget k. Its running time is $O^*(1.4969^n)$ in polynomial space [32], and $\sqrt{1.4969} \leq 1.2235$.

Independent Dominating Set. The algorithm of [7] keeps vertices free or marked; a marked vertex may not enter the solution and must still be dominated. For a chosen vertex v, some free $u \in N[v]$ is in the solution, and the algorithm branches over these u, removes $N[u]$ and lowers the budget by one, marking the vertices tried in earlier branches. Its refinements choose which branchings to apply on vertices of small degree and keep this form. Its running time is $O^*(2^{0.424n})$ in polynomial space, and $2^{0.212} \leq 1.1583$.

Edge Dominating Set and Minimum Maximal Matching. Algorithm 4 of [33], in the numbering of its technical report, splits the vertices into a vertex cover C, an independent set I and undecided vertices U. A clique component H of $G[U]$ is handled by one call on the graph with a new vertex joined to H, whose optimum is exactly one larger, so the call has budget $k+1$; we use this rule for cliques of every size, which leaves the analysis unchanged since the measure counts only U. The new vertices keep the graph at most $2n$ vertices. Otherwise it branches on an undecided vertex, into I with its undecided neighbours into C, or into C, and on the paths and cycles of $G[U]$ into two or a constant number of children. At a leaf C is a vertex cover, and the smallest edge dominating set whose endpoints include C is computed through a maximum matching. Unweighted Minimum Maximal Matching has the same decision problem by the conversion in [33]. For the weighted versions, Algorithms 5 and 7 of [33] put isolated undecided vertices into I with budget k, and handle cliques of two or three vertices by one call: for weighted edge domination with the budget raised by the weight of the new edge, for weighted minimum maximal matching with zero-weight new edges and budget k. Larger cliques are branched on. Their leaves are a generalized edge cover or a perfect matching, both computed in polynomial time. The running time is $O^*(1.3226^n)$ in polynomial space, and $\sqrt{1.3226} \leq 1.1501$. □

The faster algorithms of [23] for Dominating Set, of [6] for Independent Dominating Set and of [42] for Edge Dominating Set and unweighted Minimum Maximal Matching would give the bounds $O(1.2192^n)$, $O(1.1555^n)$ and $O(1.1472^n)$ by the same argument if they are backtracking trees in this sense. Their structure is not covered here.

6 Monotone local search

Monotone local search [17] turns an algorithm that extends a partial solution into an exact algorithm. An implicit set system Φ maps an instance I of size N to a universe U_I of n elements and a family F_I of subsets, with membership decidable in polynomial time. The problem Φ-Subset asks for a set in F_I. The problem Φ-Extension asks, for $X \subseteq U_I$ and an integer k, whether some $S \subseteq U_I \setminus X$ with $|S| \leq k$ has $X \cup S \in F_I$. If Φ-Extension is solved in time $c^k N^{O(1)}$, then Φ-Subset is solved by a randomized algorithm in time $(2-1/c)^n N^{O(1)}$ [17].

Theorem 6.1. *Let $c > 1$, and suppose that Φ-Extension is decided by a backtracking tree with at most $p(N)\,c^k$ nodes, for a fixed polynomial p. Then Φ-Subset has a quantum algorithm with bounded error, polynomial space and no quantum random access memory, in time $(2-1/c)^{n/2} N^{O(1)}$. The same holds if Φ-Extension has a bounded-error quantum algorithm in polynomial space whose running time on every input is at most $c^{k/2} p(N)$, and in the permissive form where the tree, on*

(I, X, k), accepts whenever some S with $|S| \le k$ has $X \cup S \in F_I$, and accepts only if some S of any size has $X \cup S \in F_I$.

Proof. Fix $k' \le n$, and let $t \le k'$ minimize $\binom{n}{t}\binom{k'}{t}^{-1} c^{k'-t}$. Prepare the uniform superposition over the subsets $Y \subseteq U_I$ of size t, which takes polynomially many gates. On each Y, run the walk of [30] on the tree of $(I, Y, k'-t)$ with the bound $T = p(N)\, c^{k'-t}$, with its error reduced to δ by a majority over $O(\log 1/\delta)$ runs and with measurements deferred, and write the answer into one qubit. This oracle takes time $\sqrt{T}\,\mathrm{poly}(N)$, the same for every Y. Let $p_0 = \binom{k'}{t}\binom{n}{t}^{-1}$ and $m = \lceil p_0^{-1/2} \rceil$. Measure once without amplification and once after j rounds of amplitude amplification [3, 4], for j uniform in $\{0, \ldots, m-1\}$, and accept if a measured Y passes the extension decision again, with a majority over $O(\log n)$ runs. Do this for every $k' \le n$. The walk needs an explicit bound T on the nodes of the full tree, explored without stopping at an accepting leaf.

If F_I contains a set S of size k', every $Y \subseteq S$ with $|Y| = t$ is a yes-instance of the extension, with $S \setminus Y$ as completion, so with an exact oracle the success probability $p = \sin^2\theta$ of the start state is at least p_0. If $p > 3/4$, the measurement without amplification succeeds with probability above $3/4$. Otherwise $\sin 2\theta = 2\sqrt{p(1-p)} \ge \sqrt{p} \ge \sqrt{p_0}$, so $m \sin 2\theta \ge 1$. Then

$$\frac{1}{m}\sum_{j<m} \sin^2((2j+1)\theta) \;=\; \frac{1}{2} - \frac{\sin(4m\theta)}{4m\sin 2\theta} \;\ge\; \frac{1}{4}.$$

An oracle with error δ differs from the exact one by $O(\sqrt{\delta})$ in norm per call, so $\delta = 1/(m^2\mathrm{poly}(n))$, at a cost of $O(\log m) = O(n)$ repetitions, changes this by at most $1/100$. If F_I is empty, no sample is marked, also in the permissive form, and the final check rejects except with small probability. A constant number of repetitions gives bounded error.

The run for k' takes time $m\sqrt{T}\,\mathrm{poly}(N) = (\binom{n}{t}\binom{k'}{t}^{-1} c^{k'-t})^{1/2}\mathrm{poly}(N)$, the square root of the classical run in [17]. By Lemma 2.2 of [17], $\max_{k'} \min_t \binom{n}{t}\binom{k'}{t}^{-1} c^{k'-t} \le (2-1/c)^n n^{O(1)}$, so the total time over all $k' \le n$ is $(2-1/c)^{n/2} N^{O(1)}$. The space is that of the walk and of n sample qubits. A solution follows from an accepted Y by descending the extension tree with the walk at each child. □

Theorem 6.2. *With bounded error, polynomial space and no quantum random access memory, the problems of Table 3 have quantum algorithms with the running times listed there, up to polynomial factors.*

Proof. In each case, removing X (deleting the vertices, cutting the edges or setting the variables to one and simplifying) (for hitting sets, removing the hyperedges hit by X and lowering the weight bound by the weight of X) turns the extension instance (I, X, k) into an instance of the same problem with budget k, and we check that the algorithm is a backtracking tree whose number of nodes is bounded by its stated running time. The measure and conquer analyses prove a bound Cc^k on the leaves of the full tree by induction on the measure, with an explicit constant C, and the depth is a fixed polynomial, which gives the explicit bound T.

Feedback Vertex Set. The algorithm of [20] works on (G, F, k) with undeletable vertices F, empty at the start. Its reductions delete a vertex, bypass a vertex of degree two or lower a multiplicity, each with one call of budget k or $k-1$. A reduction to matroid matching answers in polynomial time when the maximum degree is at most three, and a pruning rule answers no. Otherwise it branches into deleting a vertex u of maximum degree, with budget $k-1$, or contracting u with its neighbours in F into F, with budget k. Every branch removes a free vertex and every reduction shrinks the

Problem	Extension algorithm	c	Classical, same extension	This paper
Feedback Vertex Set	branching [20]	3.460	1.7110^n	$\mathbf{1.3081^n}$
Cluster Vertex Deletion	branching [37]	1.811	1.4479^n	$\mathbf{1.2033^n}$
Subset and Group Feedback Vertex Set	LP-branching [21]	4	1.7500^n	$\mathbf{1.3229^n}$
Node Unique Label Cover, fixed labels Σ	LP-branching [21]	$\lvert\Sigma\rvert^2$	$(2-\lvert\Sigma\rvert^{-2})^n$	$\mathbf{(2-\lvert\Sigma\rvert^{-2})^{n/2}}$
3-Hitting Set, also weighted	branching [36]	2.168	1.5388^n	$\mathbf{1.2405^n}$
Proper Interval Vertex Deletion	branching [9]	6	1.8334^n	$\mathbf{1.3541^n}$
Multicut on Trees	branch and search [24]	$\sqrt{\sqrt{2}+1}$	1.3565^n	$\mathbf{1.1647^n}$
Min-Ones d-SAT, $d \geq 3$; d-Hitting Set, $d \geq 4$	clause branching [17]	d	$(2-1/d)^n$	$\mathbf{(2-1/d)^{n/2}}$

Table 3: Corollaries of Theorem 6.1. For Multicut on Trees, n is the number of edges. For $d = 3, 4, 5$ the last row gives 1.2910^n, 1.3229^n and 1.3417^n. The classical column applies [17] to the same extension algorithm; for d-Hitting Set [17] gives faster classical bounds through other extension algorithms, such as 1.5182^n for 3-Hitting Set.

multigraph, so the depth is polynomial. The measure of [20] is k at the start and nonnegative after pruning, so the full tree has $O^*(3.460^k)$ leaves, and $\sqrt{2-1/3.460} \leq 1.3081$.

Cluster Vertex Deletion. The reductions of [37] answer or make one call. Its branching rules branch over the twins of a vertex v and over the members of a dominating family of vertex covers of an auxiliary graph, computed by a recursion that branches on a constant number of sets in each step. This family can be large, so we expand the recursion inside the tree: each of its leaves X is followed by the call on $G-X$ with budget $k-|X|$, and the same is done for the second family built by one of the rules. The analysis of [37] bounds the branches over family members by $O^*(1.811^k)$. The failed leaves of the inner recursion, with negative budget, are not counted there, but its branching vectors are at least $(1, 2)$, so each inner recursion has $O^*(1.619^k) \leq O^*(1.811^k)$ leaves and the whole tree has at most a polynomial factor more, and $\sqrt{2-1/1.811} \leq 1.2033$.

Group and Subset Feedback Vertex Set, Node Unique Label Cover. The algorithms of [21] are LP-branching. Each node solves a half-integral relaxation in polynomial time and fixes, by single calls, assignments that do not raise the relaxed optimum. Otherwise it branches into two children for Group Feedback Vertex Set (Lemma 8 of [21]) or $|\Sigma|$ children for Unique Label Cover (Lemma 1 and Corollary 4 of [21]), and each branch lowers the gap between the budget and the relaxed optimum by at least $1/2$. So there are at most 4^k or $|\Sigma|^{2k}$ leaves. Subset Feedback Vertex Set is equivalent to its edge version, with the edges at the terminals as the edge subset, on the same graph and with the same budget [12], and the edge version reduces to Group Feedback Vertex Set on the same graph with the same budget by Lemma 5 of [11], with the group $\mathbb{Z}_2^{|S|}$ for the edge subset S. Since this group is large, the relaxation is solved in the oracle model of Section 5.2 of [21]. For Group Feedback Vertex Set $\sqrt{2-1/4} \leq 1.3229$; for Unique Label Cover $|\Sigma|$ is fixed.

3-Hitting Set and Weighted 3-Hitting Set. The algorithm of Appendix B.1 of the arXiv version of [36], for integer weights of polynomial bit length, works with a weight bound W and a size parameter k, and its rules handle hyperedges of sizes one, two and three. Its reductions answer, solving hypergraphs of degree at most two in polynomial time, or make one call, one of them with

modified weights, and each branching rule returns the first of two answers that is not NIL. A non-NIL answer is a hitting set of weight at most W, and one is returned whenever a hitting set of weight at most W and size at most k exists, which is the permissive form of Theorem 6.1. Its measure $k - \alpha(G) + 1$ with $\alpha \leq 0.87$ and branching numbers at most 2.168 give $O^*(2.168^k)$ leaves. The family F_I is the set of hitting sets within the weight bound. Unit weights give 3-Hitting Set, and $\sqrt{2 - 1/2.168} \leq 1.2405$.

Proper Interval Vertex Deletion. If G has an induced subgraph from a finite family of graphs with at most six vertices, found in linear time, the algorithm of [9] branches into deleting each of its vertices with budget $k - 1$; otherwise the problem is solved in linear time. So there are at most 6^k leaves, and $\sqrt{2 - 1/6} \leq 1.3541$. Interval Vertex Deletion is not included, since the algorithm of [10] solves components and modules to optimality before it continues.

Multicut on Trees. Cutting X removes its edges and the requests they separate and leaves a forest, which the algorithm of [24] handles, since it cuts edges itself. Its reductions are single calls, and those that use minimum vertex covers do so on auxiliary graphs of maximum degree two, in polynomial time. The branching rules make two or a constant number of children, the depth is polynomial, and the number of leaves is at most ρ^k with $\rho = \sqrt{\sqrt{2} + 1}$. Then $\sqrt{2 - 1/\rho} \leq 1.1647$.

Min-Ones d-SAT and d-Hitting Set. The algorithm of [17] picks an unsatisfied clause and branches over its at most d variables set to one, with budget $k - 1$, so there are at most d^{k+1} nodes. □

Quantum search over all subsets takes time $2^{n/2}$. For Feedback Vertex Set, the walk on the algorithm of [31] gives $O(1.3748^n)$, stated for Maximum Induced Forest in [27]. The walk on a classical exact algorithm with base b gives $\sqrt{b}$ when that algorithm is a backtracking tree. The exact bases listed in [17] give at best $\sqrt{1.7347} > 1.3170$ for Feedback Vertex Set, $\sqrt{1.6181} > 1.2720$ for Cluster Vertex Deletion, $\sqrt{1.8638} > 1.3652$ for Subset Feedback Vertex Set and $\sqrt{1.8704} > 1.3676$ for 4-Hitting Set, and [17] lists no exact algorithm faster than brute force for Group Feedback Vertex Set, Node Unique Label Cover, Multicut on Trees and Min-Ones d-SAT, and for Proper Interval Vertex Deletion only $(2-\varepsilon)^n$ with $\varepsilon < 10^{-20}$. To our knowledge, no quantum form of monotone local search has been stated before. The classical column of Table 3 uses the same extension algorithms; faster classical bounds exist for some rows, such as 1.6097^n for Min-Ones 3-SAT [17]. For 3-Hitting Set the walk on the exact algorithm of [39] would give $\sqrt{1.6278} > 1.2758$ if it is a backtracking tree, and the 2.0755^k extension algorithm of [39] would give 1.2322^n through Theorem 6.1 if it is one. The faster 2.0409^k algorithm of [38] computes the optimum of a component by recursive calls before a tail call and is not a plain OR tree, so it is not used. Problems that reduce to 3-Hitting Set on the same universe, such as Feedback Vertex Set in Tournaments, get the bound 1.2405^n as well, although their own extension algorithms may give better bounds if they are backtracking trees.

6.1 Approximation

Approximate monotone local search [13, 14] turns extension algorithms with ratio α and running time c^k into exponential-time β-approximations. Let L be a finite list of pairs (α, c) with $\alpha, c \geq 1$ and let $\beta \geq 1$. For a guess k of OPT, a pair (α, c) and $t \leq \beta k$, let $x = \max(0, \lceil (1 - \beta/\alpha)k + t/\alpha \rceil)$ and $\ell = k - x$, and let $h(n, k, t, x)$ be the probability that a uniform t-subset of $[n]$ meets $[k]$ in at least x elements. Let $\text{best}(L, \beta)$ be the base of $\max_{k \leq n/\beta} \min_{(\alpha,c),t} c^\ell / h(n, k, t, x)$, up to polynomial factors [14].

Theorem 6.3. *Let $\beta \geq 1$, and let each pair $(\alpha, c) \in L$ come with an extension procedure of one of two kinds: (a) a randomized polynomial-time algorithm that, on (I, X, ℓ) such that some S with $|S| \leq \ell$ has $X \cup S \in F_I$, returns Y with $X \cup Y \in F_I$ and $|Y| \leq \alpha\ell$ with probability at least*

$q(N,\ell) \geq c^{-\ell}/p(N)$, where q is computable in polynomial time; or (b) for $\alpha = 1$, a backtracking tree with at most $p(N)c^{\ell}$ nodes that accepts (I, X, ℓ) exactly when such an S exists, and whose accepting leaves give one. Then there is a quantum β-approximation for Φ with bounded error, polynomial space and no quantum random access memory, in time $\mathrm{best}(L,\beta)^{n/2}N^{O(1)}$.

Proof. For each $k \leq n/\beta$, take the pair and t that minimize $c^{\ell}/h(n,k,t,x)$. Prepare the uniform superposition over the t-subsets X, together with the random bits of the procedure for kind (a), whose branching probabilities are rounded to $O(\log N)$ bits at the loss of a constant factor. Mark a branch if $Z = X \cup Y \in F_I$ and $|Z| \leq \beta k$ for kind (a), or if the detection walk accepts (I, X, ℓ) for kind (b), with its error reduced as in the proof of Theorem 6.1. If $k = \mathrm{OPT}$ and $|X \cap \mathrm{OPT}| \geq x$, the rest of the optimum completes X within ℓ, and $|Z| \leq t + \alpha\ell \leq \beta k$, since $\alpha x \geq (\alpha - \beta)k + t$, or $x = 0$ and $t + \alpha k \leq \beta k$. So the marked probability is at least $q_0 = h(n,k,t,x)$ times $q(N,\ell)$ for kind (a). With $m = \lceil 1/\sqrt{q_0} \rceil$, one measurement without amplification and one after a uniform number of rounds below m succeed with probability at least $1/4$, as in the proof of Theorem 6.1. For kind (b), Y is found by descending the tree. The run costs $\sqrt{c^{\ell}/h(n,k,t,x)}\,\mathrm{poly}(N)$, the square root of the classical cost, and the smallest checked set over all k is returned. □

The randomized branching algorithms of [25] for Vertex Cover and 3-Hitting Set are polynomial-time procedures of kind (a) that succeed with probability at least $(c^* + \varepsilon)^{-k}$ for every $\varepsilon > 0$ and large k, where the numerically evaluated bases c^* are rounded up to the tabulated c. If the hypothesis holds only in this form, the proof gives $(\mathrm{best}(L,\beta) + \varepsilon)^{n/2}$ for every $\varepsilon > 0$, and since best is continuous in the bases, the rounded-up entries below still hold. Polynomial-time approximations are of kind (a) with $c = 1$: a maximal matching for Vertex Cover, a maximal family of disjoint hyperedges for d-Hitting Set, and the algorithms of [2, 16, 1, 40, 22] for Feedback Vertex Set, Subset Feedback Vertex Set, Cluster Vertex Deletion, Proper Interval Vertex Deletion and Multicut on Trees, with ratios $2, 8, 2, 6, 2$. The trees of Theorem 6.2 are of kind (b), since they count their budget and accept only within it; for 3-Hitting Set we use unit weights. Table 4 gives the resulting bounds.

Problem	$\beta = 1.1$	$\beta = 1.2$	$\beta = 1.5$	$\beta = 1.8$	exact, quantum
Vertex Cover	$\mathbf{1.0555^n}$	$\mathbf{1.0312^n}$	$\mathbf{1.0050^n}$	$\mathbf{1.0003^n}$	1.0953^n
3-Hitting Set	$\mathbf{1.1607^n}$	$\mathbf{1.1135^n}$	$\mathbf{1.0458^n}$	$\mathbf{1.0211^n}$	1.2405^n
4-Hitting Set	$\mathbf{1.2288^n}$	$\mathbf{1.1861^n}$	$\mathbf{1.1233^n}$	$\mathbf{1.0933^n}$	1.3229^n
Feedback Vertex Set	$\mathbf{1.2161^n}$	$\mathbf{1.1752^n}$	$\mathbf{1.0987^n}$	$\mathbf{1.0315^n}$	1.3081^n
Subset Feedback Vertex Set	$\mathbf{1.2288^n}$	$\mathbf{1.1861^n}$	$\mathbf{1.1233^n}$	$\mathbf{1.0933^n}$	1.3229^n
Cluster Vertex Deletion	$\mathbf{1.1318^n}$	$\mathbf{1.1047^n}$	$\mathbf{1.0676^n}$	$\mathbf{1.0315^n}$	1.2033^n
Proper Interval Vertex Deletion	$\mathbf{1.2558^n}$	$\mathbf{1.2097^n}$	$\mathbf{1.1404^n}$	$\mathbf{1.1066^n}$	1.3541^n
Multicut on Trees	$\mathbf{1.1034^n}$	$\mathbf{1.0817^n}$	$\mathbf{1.0525^n}$	$\mathbf{1.0315^n}$	1.1647^n

Table 4: Quantum β-approximation by Theorem 6.3, with bounded error, polynomial space and no quantum random access memory, rounded up. The last column is the quantum exact bound of Table 3, and for Vertex Cover the bound through Maximum Independent Set of [34]. For Multicut on Trees, n is the number of edges.

Every quantum exact algorithm is also a β-approximation, and every entry is below the quantum exact bound. To our knowledge, no quantum algorithm designed for exponential-time approximation of these problems has been stated before. Amplitude amplification over random sets of size $\beta \cdot \mathrm{OPT}$, the quantum form of the brute-force approximation of [13], gives 1.3097^n, 1.2580^n, 1.1769^n and 1.1360^n for the four ratios. The classical bounds of [13, 14] are the squares of the table entries or smaller, since they also use extension algorithms not covered by Theorem 6.3, such as the randomized 2.69998^k algorithm for Feedback Vertex Set.

The weighted form of approximate monotone local search [15] groups the elements into classes of weights within a factor $1+\delta/2$, puts all classes far below the heaviest class of the optimum into the sample, and runs the unweighted scheme in each of the $O(\delta^{-1}\log(n/\delta))$ remaining classes with one extension procedure. Sampling in every class inside one amplitude amplification, with the weight bound τ computed from the guesses as the marking threshold, gives the square root in the same way, with a subexponential factor for the guesses. The weighted Vertex Cover algorithm of [36] in polynomial space, with base 1.381, returns the first answer that is not NIL, so it is a tree of kind (b). This gives a weighted 1.1-approximation for Vertex Cover in $O(1.0788^n)$ and, through Theorem 6.1, exact weighted Vertex Cover in $O(1.1296^n)$; weighted 3-Hitting Set gets $O(1.1607^n)$ at $\beta = 1.1$.

7 Verification

The Lean library, built on Mathlib with only the standard axioms, proves the inequality $s\,x^{2(s-1)} \le y^{2s} \Rightarrow s^{1/(2s)}x'^{(s-1)/s} \le y$ for $0 \le x' \le x$, and the rows $k = 3, 4, 5$ above. Not formalized are the backtracking structure of [28], the quantum walk and tree size estimation, Theorem 9 of [35], Proposition 4.1 and the rows $k = 6$ to 20.

For the domination problems, the Lean files check $\sqrt{1.4969} \le 1.2235$, $\sqrt{1.4969^n} \le 1.2235^n$, $\sqrt{1.3226} \le 1.1501$ and $2^{53/250} \le 1.1583$. Scripts run the decision procedures of Section 5: Algorithm 4 of [32] with all its rules, the basic branching of [7] with marking, and Algorithm 2 of [33], which has the same kinds of steps as its Algorithm 4. On 400, 400 and 300 random graphs, the smallest accepted budget equals the optimum computed by brute force in every case. The scripts run the unweighted procedures; the refined branchings of [7] and of Algorithm 4 of [33], the weighted rules and the bounds on the tree size are not run.

For Section 6, the Lean files prove the identity $2\sin x \sum_{j<m}\cos((2j+1)x) = \sin(2mx)$, the averaging bound $\sum_{j<m}\sin^2((2j+1)\theta) \ge m/4$ for $m\sin 2\theta \ge 1$, and the bases of Table 3. A script checks $\max_k \min_t \binom{n}{t}\binom{k}{t}^{-1} c^{k-t} \le (2-1/c)^n$ for $n \in \{50, 200, 1000, 3000\}$ and each c of the table. A second script runs the sampling and amplification with an exact classical oracle on random instances of 3-Hitting Set, Feedback Vertex Set and Cluster Vertex Deletion with up to 10 elements: clause branching, the branching of [20] with its matroid matching step replaced by further branching, and branching on induced paths with three vertices. On 120 instances each, the smallest accepted budget equals the optimum, at least $\binom{k'}{t}$ samples are marked at $k' = \mathrm{OPT}$, the averaged success probability is at least 0.417, and state-vector runs agree with the closed form. The walk, the bounded-error oracle, the algorithms of [37, 24, 21, 36, 9] and the tree sizes are not run. For Section 6.1, a script computes $\mathrm{best}(L, \beta)$ from the large-deviation exponent of the hypergeometric tail, reproduces the values 1.2072, 1.2099, 1.2068 and 1.4156 of [14] and 1.114 of [13], and checks the exact cost at $n = 1000$ against $\mathrm{best}(L, \beta)^n$ for every entry of Table 4; the maximization over k/n is numerical.

A classical version of the colouring recursion, with OR in place of quantum search, is compared with brute force on random graphs with 5 to 9 vertices and $3 \le k \le 8$. Its base case respects the size limits strictly, and the limits passed on are kept tight or enlarged at random. With either reduction chosen at random, and with Reduction 2 only, it agrees with brute force on 300 graphs each. As a control, lowering the limit for $G - S$ by one makes 143 of 300 answers wrong, so the test does depend on the limits.

8 Discussion

The gain comes from the classical algorithm of [28] and carries through the reductions of [8, 35] to every $k \le 20$. Any further classical improvement of 3-colouring by a branching algorithm carries

over in the same way, through the same computation. The exponents for $k \geq 6$ are upper bounds for the explicit procedure of Proposition 4.1. With finer grids and pieces they approach the values of the recursion (1), (2), which we expect but do not prove.

For the domination problems the quantum bounds are the square roots of the classical ones. They follow once the decision version of each classical algorithm is seen to be a backtracking tree, and any faster classical algorithm of this kind gives a faster quantum one in the same way.

For monotone local search the quantum bound is the square root of the classical one whenever the extension algorithm is a backtracking tree. Extension algorithms with steps that compute the optimum of a part before a tail call, such as the 2.0409^k algorithm for 3-Hitting Set [38], and the multivariate and sample-dependent forms of monotone local search, are not covered here. The same holds for approximate monotone local search, where randomized extension procedures that are not polynomial-time procedures, and deterministic approximate branching algorithms, are not covered.

Use of AI

The proofs, the code and the Lean files were developed with Claude Opus 5.5 (Anthropic), which was also used to search the literature, to check the proofs and to draft the text. The author is responsible for the content.

References

[1] M. Aprile, M. Drescher, S. Fiorini, T. Huynh. A tight approximation algorithm for the cluster vertex deletion problem. Mathematical Programming 197(2), 2023.

[2] V. Bafna, P. Berman, T. Fujito. A 2-approximation algorithm for the undirected feedback vertex set problem. SIAM Journal on Discrete Mathematics 12(3), 1999.

[3] M. Boyer, G. Brassard, P. Høyer, A. Tapp. Tight bounds on quantum searching. Fortschritte der Physik 46, 1998.

[4] G. Brassard, P. Høyer, M. Mosca, A. Tapp. Quantum amplitude amplification and estimation. Contemporary Mathematics 305, 2002.

[5] R. Beigel, D. Eppstein. 3-coloring in time $O(1.3289^n)$. Journal of Algorithms 54(2), 2005, arXiv:cs/0006046.

[6] N. Bourgeois, F. Della Croce, B. Escoffier, V. Th. Paschos. Fast algorithms for min independent dominating set. Discrete Applied Mathematics 161(4-5), 2013.

[7] N. Bourgeois, B. Escoffier, V. Th. Paschos. Fast algorithms for min independent dominating set. SIROCCO 2010, arXiv:0905.1993.

[8] J. M. Byskov. Enumerating maximal independent sets with applications to graph colouring. Operations Research Letters 32(6), 2004.

[9] Y. Cao. Unit interval editing is fixed-parameter tractable. ICALP 2015, arXiv:1504.04470.

[10] Y. Cao. Linear recognition of almost interval graphs. SODA 2016, arXiv:1403.1515.

[11] M. Cygan, M. Pilipczuk, M. Pilipczuk. On group feedback vertex set parameterized by the size of the cutset. Algorithmica 74, 2016, arXiv:1112.6255.

[12] M. Cygan, M. Pilipczuk, M. Pilipczuk, J. O. Wojtaszczyk. Subset feedback vertex set is fixed-parameter tractable. ICALP 2011.

[13] B. C. Esmer, A. Kulik, D. Marx, D. Neuen, R. Sharma. Faster exponential-time approximation algorithms using approximate monotone local search. ESA 2022, arXiv:2206.13481.

[14] B. C. Esmer, A. Kulik, D. Marx, D. Neuen, R. Sharma. Optimally repurposing existing algorithms to obtain exponential-time approximations. SODA 2024, arXiv:2306.15331.

[15] B. C. Esmer, A. Kulik, D. Marx, D. Neuen, R. Sharma. Approximate monotone local search for weighted problems. IPEC 2023, arXiv:2308.15306.

[16] G. Even, J. Naor, L. Zosin. An 8-approximation algorithm for the subset feedback vertex set problem. SIAM Journal on Computing 30(4), 2000.

[17] F. V. Fomin, S. Gaspers, D. Lokshtanov, S. Saurabh. Exact algorithms via monotone local search. Journal of the ACM 66(2), 2019, arXiv:1512.01621.

[18] M. Fürer. Solving NP-complete problems with quantum search. LATIN 2008.

[19] S. Gaspers, J. Z. Li. Quantum algorithms for graph coloring and other partitioning, covering, and packing problems. ICALP 2024, arXiv:2311.08042.

[20] Y. Iwata, Y. Kobayashi. Improved analysis of highest-degree branching for feedback vertex set. IPEC 2019, arXiv:1905.12233.

[21] Y. Iwata, M. Wahlström, Y. Yoshida. Half-integrality, LP-branching and FPT algorithms. SIAM Journal on Computing 45(4), 2016, arXiv:1310.2841.

[22] N. Garg, V. V. Vazirani, M. Yannakakis. Primal-dual approximation algorithms for integral flow and multicut in trees. Algorithmica 18, 1997.

[23] Y. Iwata. A faster algorithm for dominating set analyzed by the potential method. IPEC 2011, LNCS 7112.

[24] I. Kanj, G. Lin, T. Liu, W. Tong, G. Xia, J. Xu, B. Yang, F. Zhang, P. Zhang, B. Zhu. Algorithms for cut problems on trees. COCOA 2014, LNCS 8881, arXiv:1304.3653.

[25] A. Kulik, H. Shachnai. Analysis of two-variable recurrence relations with application to parameterized approximations. FOCS 2020, arXiv:1911.02653.

[26] E. L. Lawler. A note on the complexity of the chromatic number problem. Information Processing Letters 5(3), 1976.

[27] Kvantu algoritms maksimāla inducēta meža problēmai (a quantum algorithm for the maximum induced forest problem). Thesis, University of Latvia, `https://dspace.lu.lv/handle/7/66169`.

[28] L. Meijer. 3-Coloring in time $O(1.3217^n)$. arXiv:2302.13644, 2023.

[29] J. W. Moon, L. Moser. On cliques in graphs. Israel Journal of Mathematics 3, 1965.

[30] A. Montanaro. Quantum-walk speedup of backtracking algorithms. Theory of Computing 14(15), 2018, arXiv:1509.02374.

[31] I. Razgon. Exact computation of maximum induced forest. SWAT 2006, LNCS 4059.

[32] J. M. M. van Rooij, H. L. Bodlaender. Exact algorithms for dominating set. Discrete Applied Mathematics 159(17), 2011. Technical report UU-CS-2009-025.

[33] J. M. M. van Rooij, H. L. Bodlaender. Exact algorithms for edge domination. Algorithmica 64(4), 2012. Technical report UU-CS-2007-051.

[34] S. Senge. Quantum vertex cover. Research note, `https://github.com/ssenge/LAIF/tree/main/quantum-vertex-cover`, 2026.

[35] K. Shimizu, R. Mori. Exponential-time quantum algorithms for graph coloring problems. Algorithmica 84, 2022, arXiv:1907.00529.

[36] H. Shachnai, M. Zehavi. A multivariate approach for weighted FPT algorithms. ESA 2015; arXiv:1407.2033, A multivariate framework for weighted FPT algorithms.

[37] D. Tsur. Faster parameterized algorithm for cluster vertex deletion. arXiv:1901.07609.

[38] D. Tsur. Faster parameterized algorithm for 3-hitting set. arXiv:2501.06452.

[39] M. Wahlström. Algorithms, measures and upper bounds for satisfiability and related problems. PhD thesis, Linköping University, 2007.

[40] P. van 't Hof, Y. Villanger. Proper interval vertex deletion. Algorithmica 65(4), 2013.

[41] R. Wong, W.-L. Chang, W.-Y. Chung, A. V. Vasilakos. Biomolecular and quantum algorithms for the dominating set problem in arbitrary networks. Scientific Reports 13, 4205, 2023.

[42] M. Xiao, H. Nagamochi. A refined exact algorithm for edge dominating set. Theoretical Computer Science 560, 2014.